ASSIGNMENT 12.5

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.

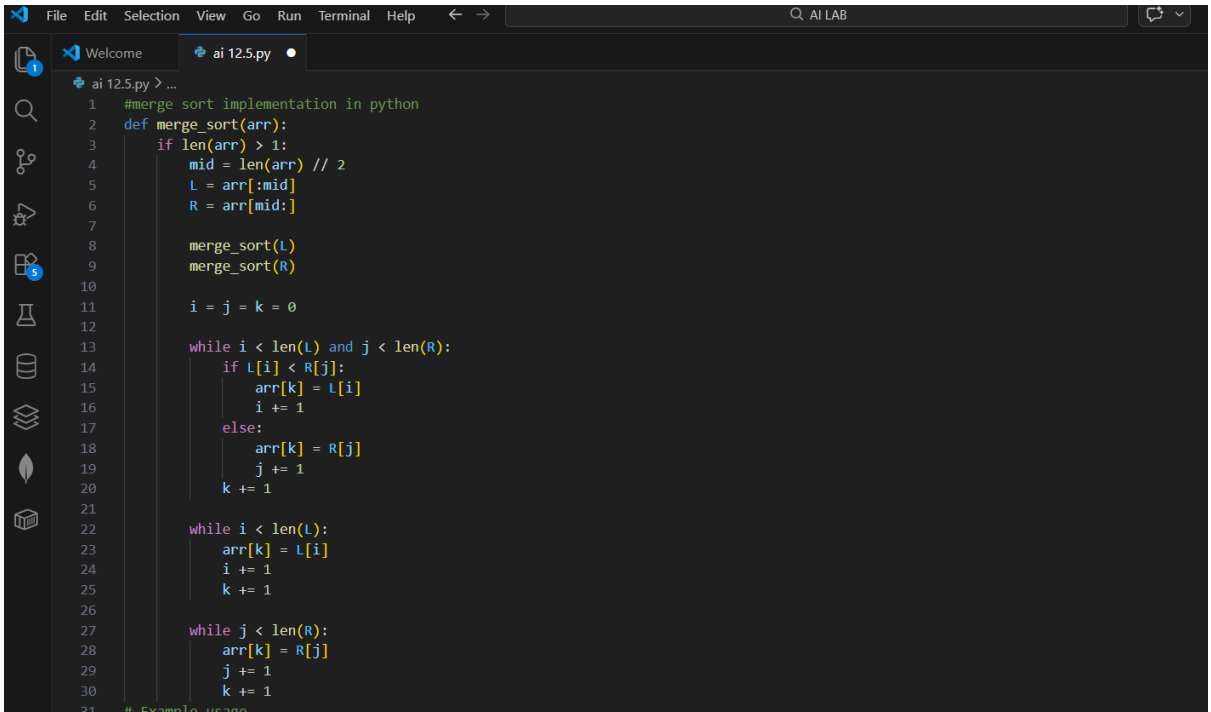
o Ask AI to include time complexity and space complexity in the function docstring.

o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

CODE:




```python
#merge sort implementation in python
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0

        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1

        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1

        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1
# Example usage
```

Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.

CODE:

```python

#binary search implementation in python
def binary_search(arr, x):
    l, r = 0, len(arr) - 1
    while l <= r:
        mid = l + (r - l) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            l = mid + 1
        else:
            r = mid - 1
    return -1
# Example usage
arr = [2, 4, 6, 8, 10, 12, 14]
x = 10
result = binary_search(arr, x)
if result != -1:
    print("Element is present at index", str(result))
else:
    print("Element is not present in array")
```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

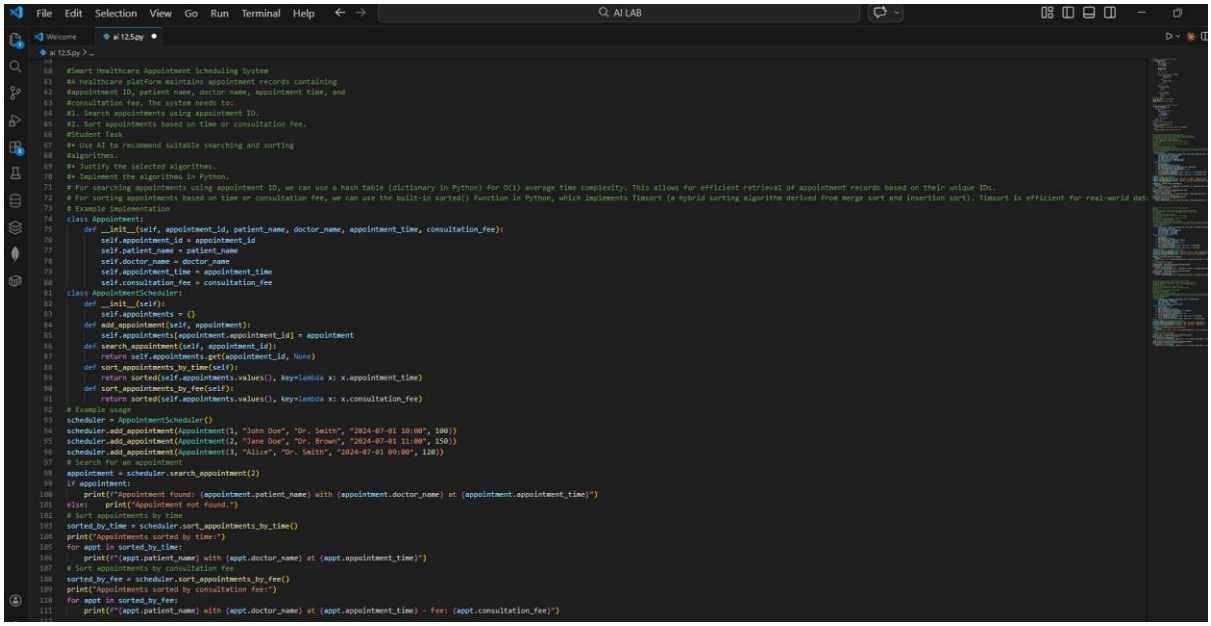
Student Task

• Use AI to recommend suitable searching and sorting algorithms.

• Justify the selected algorithms.

• Implement the algorithms in Python.

CODE:



```python
#Smart Healthcare Appointment Scheduling System
#A healthcare platform maintains appointment records containing
#appointment ID, patient name, doctor name, appointment time, and
#consultation fee. The system needs to:
#1. Search appointments using appointment ID.
#2. Sort appointments based on time or consultation fee.
#Student Task
#• Use AI to recommend suitable searching and sorting
#algorithms.
#• Justify the selected algorithms.
#• Implement the algorithms in Python.
# For searching appointments using appointment ID, we can use a hash table (dictionary in Python) for O(1) average time complexity. This allows for efficient retrieval of appointment records based on their unique IDs.
# For sorting appointments based on time or consultation fee, we can use the built-in sorted() function in Python, which implements Timsort (a hybrid sorting algorithm derived from merge sort and insertion sort). Timsort is efficient for real-world dat
# Example implementation
class Appointment:
    def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
        self.appointment_id = appointment_id
        self.patient_name = patient_name
        self.doctor_name = doctor_name
        self.appointment_time = appointment_time
        self.consultation_fee = consultation_fee
class AppointmentScheduler:
    def __init__(self):
        self.appointments = {}
    def add_appointment(self, appointment):
        self.appointments[appointment.appointment_id] = appointment
    def search_appointment(self, appointment_id):
        return self.appointments.get(appointment_id, None)
    def sort_appointments_by_time(self):
        return sorted(self.appointments.values(), key=lambda x: x.appointment_time)
    def sort_appointments_by_fee(self):
        return sorted(self.appointments.values(), key=lambda x: x.consultation_fee)
# Example usage
scheduler = AppointmentScheduler()
scheduler.add_appointment(Appointment(1, "John Doe", "Dr. Smith", "2024-07-01 10:00", 100))
scheduler.add_appointment(Appointment(2, "Jane Doe", "Dr. Brown", "2024-07-01 11:00", 150))
scheduler.add_appointment(Appointment(3, "Alice", "Dr. Smith", "2024-07-01 09:00", 120))
# Search for an appointment
appointment = scheduler.search_appointment(2)
if appointment:
    print(f"Appointment found: {appointment.patient_name} with {appointment.doctor_name} at {appointment.appointment_time}")
else:    print("Appointment not found.")
# Sort appointments by time
sorted_by_time = scheduler.sort_appointments_by_time()
print("Appointments sorted by time:")
for appt in sorted_by_time:
    print(f"{appt.patient_name} with {appt.doctor_name} at {appt.appointment_time}")
# Sort appointments by consultation fee
sorted_by_fee = scheduler.sort_appointments_by_fee()
print("Appointments sorted by consultation fee:")
for appt in sorted_by_fee:
    print(f"{appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} - Fee: {appt.consultation_fee}")
```

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Student Task

• Identify efficient algorithms using AI assistance.

• Justify the algorithm choices.

• Implement searching and sorting in Python

CODE:

```python
#Task Description #4: Railway Ticket Reservation System
#Scenario
#A railway reservation system stores booking details such as ticket
#ID, passenger name, train number, seat number, and travel date. The
#system must:
#1. Search tickets using ticket ID.
#2. Sort bookings based on travel date or seat number.
#Student Task
#• Identify efficient algorithms using AI assistance.
#• Justify the algorithm choices.
#• Implement searching and sorting in Python.
# For searching tickets using ticket ID, we can use a hash table (dictionary in Python) for O(1) average time complexity. This allows for efficient retrieval of booking records based on their unique IDs.
# For sorting bookings based on travel date or seat number, we can use the built-in sorted() function in Python, which implements Timsort. Timsort is efficient for real-world data and has a time complexity of O(n log n) in the worst case.
# Example implementation
class Booking:
    def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = travel_date
class RailwayReservationSystem:
    def __init__(self):
        self.bookings = {}
    def add_booking(self, booking):
        self.bookings[booking.ticket_id] = booking
    def search_booking(self, ticket_id):
        return self.bookings.get(ticket_id, None)
    def sort_bookings_by_date(self):
        return sorted(self.bookings.values(), key=lambda x: x.travel_date)
    def sort_bookings_by_seat(self):
        return sorted(self.bookings.values(), key=lambda x: x.seat_number)
# Example usage
reservation_system = RailwayReservationSystem()
reservation_system.add_booking(Booking(1, "John Doe", "Train A", "12A", "2024-08-01"))
reservation_system.add_booking(Booking(2, "Jane Doe", "Train B", "15B", "2024-08-02"))
reservation_system.add_booking(Booking(3, "Alice", "Train A", "10A", "2024-08-01"))
# Search for a booking
booking = reservation_system.search_booking(2)
if booking:
    print(f"Booking found: {booking.passenger_name} on {booking.train_number} seat {booking.seat_number} on {booking.travel_date}")
else:
    print("Booking not found.")
# Sort bookings by travel date
sorted_by_date = reservation_system.sort_bookings_by_date()
print("Bookings sorted by travel date:")
for booking in sorted_by_date:
    print(f"{booking.passenger_name} on {booking.train_number} seat {booking.seat_number} on {booking.travel_date}")
# Sort bookings by seat number
sorted_by_seat = reservation_system.sort_bookings_by_seat()
print("Bookings sorted by seat number:")
for booking in sorted_by_seat:
    print(f"{booking.passenger_name} on {booking.train_number} seat {booking.seat_number} on {booking.travel_date}")
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

Student Task

• Use AI to suggest optimized algorithms.

• Justify the selections.

• Implement the solution in Python.

CODE:

```python
#Task Description #5: Smart Hostel Room Allocation System
#A hostel management system stores student room allocation details
#including student ID, room number, floor, and allocation date. The
#system needs to:
#1. Search allocation details using student ID.
#2. Sort records based on room number or allocation date.
#Student Task
#• Use AI to suggest optimized algorithms.
#• Justify the selections.
#• Implement the solution in Python.
# For searching allocation details using student ID, we can use a hash table (dictionary in Python) for O(1) average time complexity. This allows for efficient retrieval of allocation records based on their unique IDs.
# For sorting records based on room number or allocation date, we can use the built-in sorted() function in Python, which implements Timsort. Timsort is efficient for real-world data and has a time complexity of O(n log n) in the worst case.
# Example implementation
class Allocation:
    def __init__(self, student_id, room_number, floor, allocation_date):
        self.student_id = student_id
        self.room_number = room_number
        self.floor = floor
        self.allocation_date = allocation_date
class HostelRoomAllocationSystem:
    def __init__(self):
        self.allocations = {}
    def add_allocation(self, allocation):
        self.allocations[allocation.student_id] = allocation
    def search_allocation(self, student_id):
        return self.allocations.get(student_id, None)
    def sort_allocations_by_room(self):
        return sorted(self.allocations.values(), key=lambda x: x.room_number)
    def sort_allocations_by_date(self):
        return sorted(self.allocations.values(), key=lambda x: x.allocation_date)
# Example usage
allocation_system = HostelRoomAllocationSystem()
allocation_system.add_allocation(Allocation(1, "101", "1st Floor", "2024-09-01"))
allocation_system.add_allocation(Allocation(2, "102", "1st Floor", "2024-09-02"))
allocation_system.add_allocation(Allocation(3, "201", "2nd Floor", "2024-09-01"))
# Search for an allocation
allocation = allocation_system.search_allocation(2)
if allocation:
    print(f"Allocation found: Student ID {allocation.student_id} in room {allocation.room_number} on {allocation.allocation_date}")
else:
    print("Allocation not found.")
# Sort allocations by room number
sorted_by_room = allocation_system.sort_allocations_by_room()
print("Allocations sorted by room number:")
for allocation in sorted_by_room:
    print(f"Student ID {allocation.student_id} in room {allocation.room_number} on {allocation.allocation_date}")
# Sort allocations by allocation date
sorted_by_date = allocation_system.sort_allocations_by_date()
print("Allocations sorted by allocation date:")
for allocation in sorted_by_date:
    print(f"Student ID {allocation.student_id} in room {allocation.room_number} on {allocation.allocation_date}")
```